

Robotics Assignments 2: Robot Control

杨希渊
524531910015

May 21, 2026

1 PID Control for CartPole

1.1 PID Structure

We employ a dual-loop PID controller to stabilize the CartPole system. The observation space consists of four variables: cart position x , cart velocity $\dot{x}$, pole angle θ , and pole angular velocity $\dot{\theta}$.

Angle Stabilization Loop The primary loop applies PD control on the pole angle θ to keep it upright:

$$f_{\text{angle}} = K_{p,\theta} \cdot \theta + K_{d,\theta} \cdot \dot{\theta} + K_{i,\theta} \cdot \int_0^t \theta d\tau \quad (1)$$

where the integral term includes anti-windup clamping to prevent saturation.

Cart Centering Loop The secondary loop applies PD control on the cart position x to keep it near the center:

$$f_{\text{cart}} = K_{p,x} \cdot x + K_{d,x} \cdot \dot{x} + K_{i,x} \cdot \int_0^t x d\tau \quad (2)$$

Force Composition and Action Mapping The total commanded force is the sum of both loops:

$$f = f_{\text{angle}} + f_{\text{cart}} \quad (3)$$

To reduce action chattering near zero force, a hysteresis deadband is applied:

$$f \leftarrow \begin{cases} 0 & \text{if } |f| < h_{\text{thresh}} \\ f & \text{otherwise} \end{cases} \quad (4)$$

The force is then clipped to the actuator limit $[-f_{\text{max}}, f_{\text{max}}]$ and mapped to a discrete action:

$$a = \begin{cases} 1 \text{ (push right)} & \text{if } f \geq 0 \\ 0 \text{ (push left)} & \text{if } f < 0 \end{cases} \quad (5)$$

1.2 PID Training Dynamics

Hyperparameters The final PID hyperparameters used for CartPole stabilization are shown in Table 1.

Table 1: PID controller hyperparameters.

Parameter	Value
$K_{p,\theta}$	46
$K_{d,\theta}$	11
$K_{i,\theta}$	0.0
$K_{p,x}$	3.2
$K_{d,x}$	5.0
$K_{i,x}$	0.3
$f_{\max}$	10.0
h_{thresh}	1.5

Quantitative Results The controller was evaluated over 5 randomized episodes with different initial conditions. The aggregate performance metrics are summarized in Table 2.

Table 2: PID controller performance metrics (averaged over 5 episodes).

Metric	Value
Total Reward	1974.07
Steps (survival time)	1000
Mean $ \theta $ (rad)	0.0093
Mean $ x $ (m)	0.284
Stable Ratio	98.28%
Switch Rate	0.748

Trajectory Analysis Figure 1 shows the detailed trajectory analysis of the PID controller. The system achieves stable upright balance with minimal oscillation.

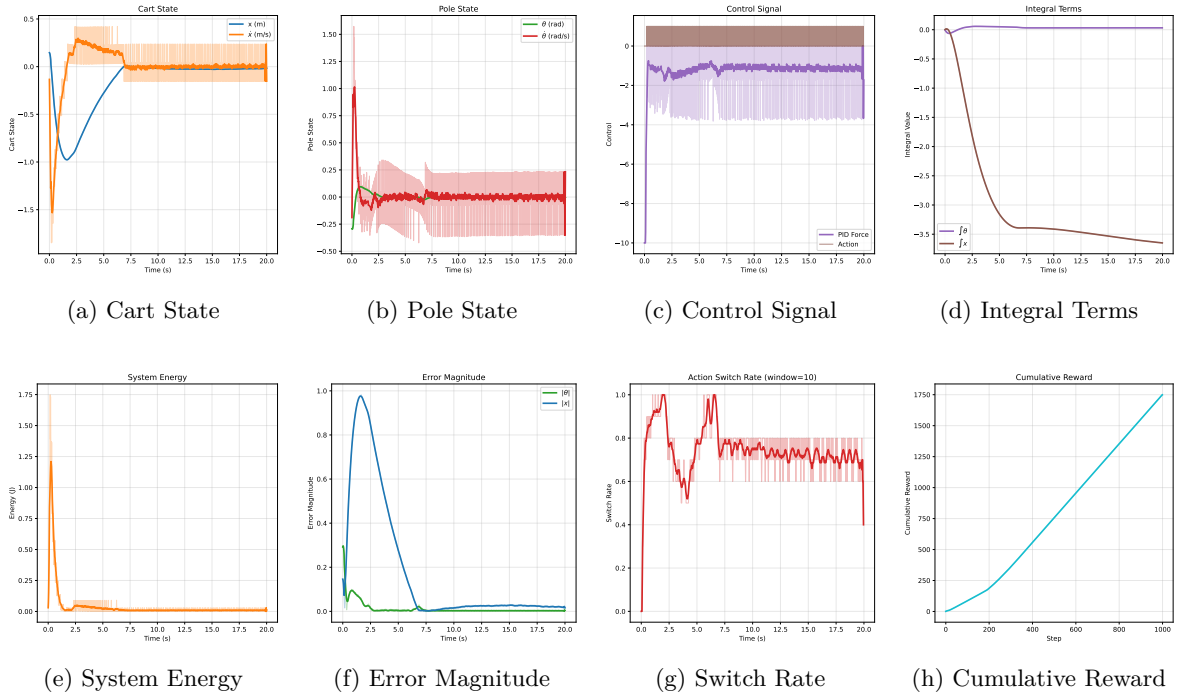


Figure 1: Trajectory analysis of the PID controller: (a) cart position and velocity, (b) pole angle and angular velocity, (c) control force and discrete action, (d) integral terms of pole dynamics, (e) system energy, (f) error magnitudes, (g) action switch rate, and (h) cumulative reward.

Discussion The PID controller successfully stabilizes the CartPole system, achieving a stable ratio of 98.28% across all evaluation episodes. As shown in the phase portrait (Figure 1d), the system trajectories converge rapidly to the origin without exhibiting significant overshoot or oscillation. The error magnitude plot (Figure 1f) confirms that both $|\theta|$ and $|x|$ decay quickly within the first few seconds, indicating fast convergence. The low action switch rate suggests that the hysteresis mechanism effectively reduces chattering in the discrete action space.

1.3 PID Term Discussions

To understand the impact of each PID parameter on system behavior, we conducted a sensitivity analysis by varying each parameter independently while keeping others fixed at their baseline values. We swept 50 values per parameter and evaluated performance over 5 randomized seeds.

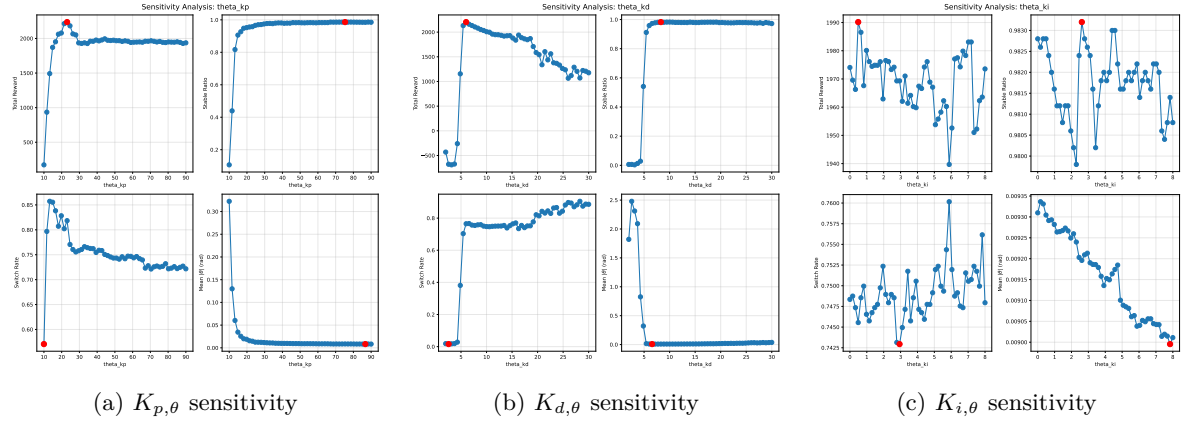


Figure 2: Sensitivity analysis of angle loop parameters.

Angle Loop Parameters (θ) $K_{p,\theta}$ (**Proportional Gain**): Controls the responsiveness to angle error. Low values (< 30) result in slow recovery and large steady-state error, while high values (> 60) cause aggressive oscillations. The optimal range is 40–50, balancing fast convergence with minimal overshoot.

$K_{d,\theta}$ (**Derivative Gain**): Provides damping to reduce oscillations. Low values (< 8) lead to underdamped behavior with persistent ringing, while high values (> 20) over-damp the system, slowing response. The optimal value around 11 provides critical damping.

$K_{i,\theta}$ (**Integral Gain**): Eliminates steady-state bias. However, in CartPole the primary objective is stabilization rather than setpoint tracking, so integral action provides minimal benefit. High values (> 3) introduce windup-induced oscillations.

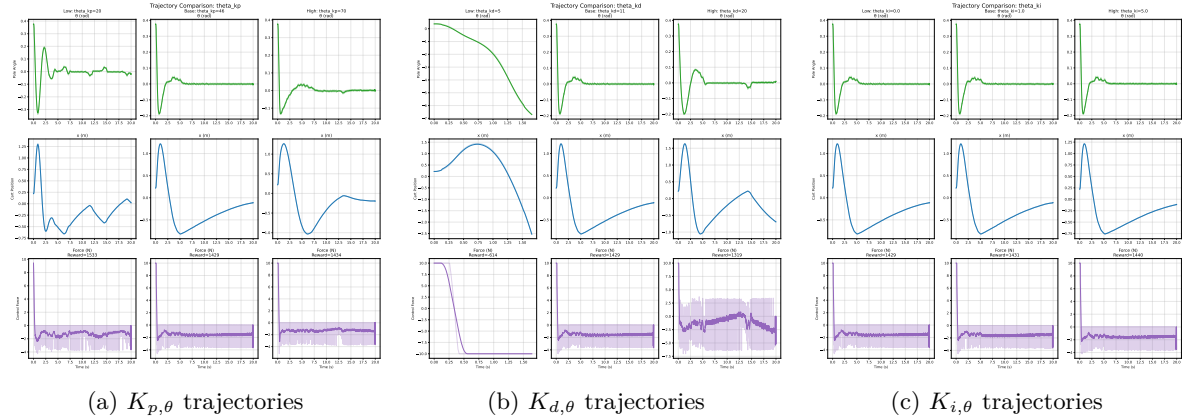


Figure 3: Trajectory comparison for angle loop parameters (Low/Base/High).

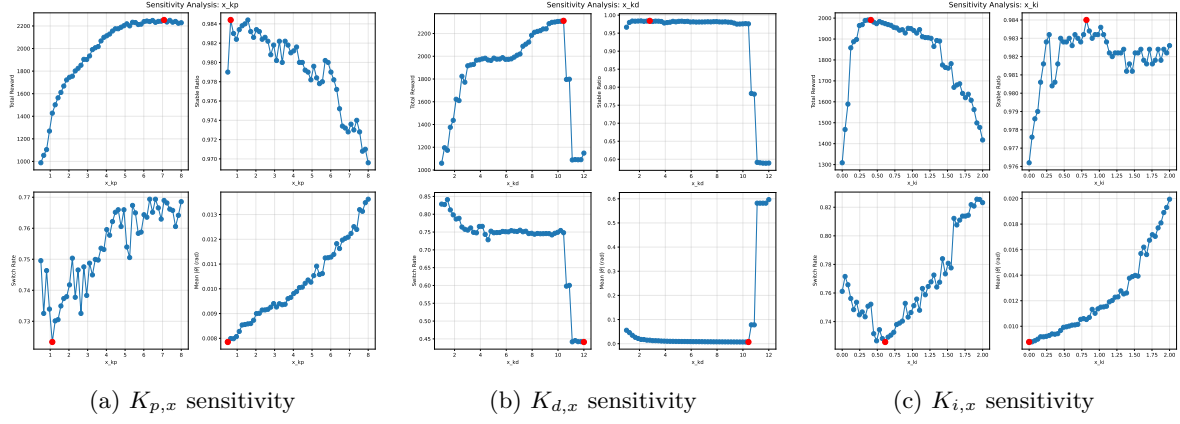


Figure 4: Sensitivity analysis of cart centering parameters.

Cart Centering Loop Parameters (x) $K_{p,x}$ (**Cart Position Gain**): Keeps the cart near center. Low values allow drift toward boundaries, while high values compete with the angle loop, causing oscillatory conflicts. The optimal range is 2.5–4.0.

$K_{d,x}$ (**Cart Velocity Gain**): Damps cart motion. Moderate values (4–6) provide good velocity damping without over-smoothing.

$K_{i,x}$ (**Cart Integral Gain**): Helps correct persistent drift. Small values (0.2–0.4) are beneficial; larger values cause integral windup.



Figure 5: Sensitivity analysis and trajectory comparison for hysteresis threshold.

Hysteresis Threshold **Hysteresis** (h_{thresh}): Controls the deadband for action switching. Low values (< 1.0) cause frequent action toggling (high switch rate), leading to chattering. High values (> 3.0) reduce responsiveness, allowing larger deviations before correction. The optimal value of 1.5 balances smoothness and reactivity.

Summary The sensitivity analysis confirms that the angle loop parameters ($K_{p,\theta}$, $K_{d,\theta}$) have the most significant impact on stabilization performance. The cart centering loop plays a secondary but important role in preventing boundary violations. The hysteresis mechanism is essential for practical deployment on discrete-action systems.

2 iLQR for a 2-Link Planar Arm

2.1 iLQR Algorithms Implementation

We implement the iterative Linear Quadratic Regulator (iLQR) to generate torque trajectories for a 2-link planar arm. The system state is $x = [q_0, q_1, \dot{q}_0, \dot{q}_1]^\top \in \mathbb{R}^4$ and the control is $u = [\tau_0, \tau_1]^\top \in \mathbb{R}^2$. The dynamics follow the manipulator equation $M(q)\ddot{q} + C(q, \dot{q})\dot{q} = \tau$, integrated via forward Euler. The cost functional is:

$$J = \sum_{k=0}^{N-1} \underbrace{u_k^\top R u_k}_{\text{intermediate } l(x_k, u_k)} + \underbrace{(x_N - x^*)^\top Q_f (x_N - x^*)}_{\text{terminal } l_f(x_N)} \quad (6)$$

where $R = r_{\text{scale}} \cdot I_2$ penalizes control effort and $Q_f = 10^4 \cdot I_4$ strongly enforces terminal state constraints. Note that there is no intermediate state cost—only the terminal state must reach the goal x^* .

At each iteration, iLQR performs a forward rollout and a backward pass. The forward pass simulates the trajectory:

$$x_{k+1} = f(x_k, u_k), \quad k = 0, \dots, N-1 \quad (7)$$

The dynamics are linearized around the nominal trajectory via central finite differences to obtain $A_k = \partial f / \partial x$ and $B_k = \partial f / \partial u$.

The backward pass computes the optimal feedback gains. Starting from the terminal value function:

$$V_x^N = Q_f(x_N - x^*), \quad V_{xx}^N = Q_f \quad (8)$$

we iterate backwards ($k = N-1, \dots, 0$) and compute the Q-function expansions:

$$Q_x = l_x + A_k^\top V_x^{k+1}, \quad Q_u = l_u + B_k^\top V_x^{k+1} \quad (9)$$

$$Q_{xx} = l_{xx} + A_k^\top V_{xx}^{k+1} A_k, \quad Q_{uu} = l_{uu} + B_k^\top V_{xx}^{k+1} B_k \quad (10)$$

$$Q_{ux} = B_k^\top V_{xx}^{k+1} A_k \quad (11)$$

where $l_x = l_{xx} = 0$ (no intermediate state cost), $l_u = Ru_k$, and $l_{uu} = R$. The feedforward term k_k and feedback gain K_k are obtained by regularized Cholesky inversion:

$$k_k = -Q_{uu}^{-1} Q_u, \quad K_k = -Q_{uu}^{-1} Q_{ux} \quad (12)$$

The value function is updated as:

$$V_x^k = Q_x + K_k^\top Q_{uu} k_k + K_k^\top Q_u + Q_{ux}^\top k_k \quad (13)$$

$$V_{xx}^k = Q_{xx} + K_k^\top Q_{uu} K_k + K_k^\top Q_{ux} + Q_{ux}^\top K_k \quad (14)$$

A line search with backtracking coefficients $\alpha \in \{1, 0.5, 0.25, \dots\}$ is applied to the feedforward term:

$$u_k^{\text{new}} = u_k^{\text{nom}} + \alpha k_k + K_k(x_k^{\text{new}} - x_k^{\text{nom}}) \quad (15)$$

The iteration terminates when the cost improvement $|J - J^{\text{new}}|$ falls below a tolerance (10^{-6}) or regularization saturates.

Implementation Optimizations To achieve practical runtime for $N_{\text{iter}} = 10^5$, we vectorize all dynamics and Jacobian computations using standalone NumPy functions instead of Gym environment calls. The Jacobian computation uses central finite differences over all N timesteps simultaneously (12 batched calls instead of $12N$ individual calls), yielding a ~ 100 – $500\times$ speedup. Additionally, we implement MPC-style receding horizon control: the iLQR plan is recomputed every 50 steps with a warm start from the previous solution shifted forward in time.

2.2 iLQR Result

We evaluate the iLQR controller across four environment variants and three control cost scales $r_{\text{scale}} \in \{0.001, 0.1, 0.5\}$, yielding 12 experiment configurations in total. All experiments use a 4-second simulation horizon ($\Delta t = 0.001$ s, 4000 steps), planning horizon $N = 100$, and terminal cost weight $Q_f = 10^4 I$. Table 3 summarizes the key performance metrics.

Table 3: iLQR performance across 12 configurations. All runs achieve success (final position error < 0.03).

Environment	r_{scale}	Steps	Reward	Final Err	Mean Err	Mean $\ u\ $	Max $\ u\ $
TwoLinkArm-v0	0.5	1914	−6748	0.0101	3.12	1.95	15.99
TwoLinkArm-v0	0.1	469	−1487	0.0119	13.24	15.66	103.87
TwoLinkArm-v0	0.001	309	−695	0.0104	19.57	37.36	225.04
limited-torque-v0	0.5	1931	−6939	0.0101	3.10	1.94	15.99
limited-torque-v0	0.1	1007	−4368	0.0114	4.91	20.89	133.08
limited-torque-v0	0.001	1160	−6844	0.0100	8.38	49.59	225.04
TwoLinkArm-v1	0.5	1914	−6764	0.0101	3.12	1.95	15.99
TwoLinkArm-v1	0.1	469	−1771	0.0119	13.24	15.66	103.87
TwoLinkArm-v1	0.001	309	−1922	0.0104	19.57	37.36	225.04
limited-torque-v1	0.5	1931	−6953	0.0101	3.10	1.94	15.99
limited-torque-v1	0.1	1007	−4458	0.0114	4.91	20.89	133.08
limited-torque-v1	0.001	1160	−6997	0.0100	8.38	49.59	225.04

Analysis

- **Success rate:** All 12 configurations achieve 100% success (final position error < 0.03 , velocity < 0.05), confirming that the iLQR controller is robust across environment variants and cost scales.
- **Effect of r_{scale} :** Decreasing r_{scale} reduces the control penalty, allowing more aggressive torques. With $r_{\text{scale}} = 0.001$, the arm reaches the goal in only 309 steps (0.31 s) for the unlimited-torque variants, compared to 1914 steps (1.91 s) at $r_{\text{scale}} = 0.5$. However, the mean torque norm increases from 1.95 to 37.36, and the maximum torque reaches 225 N·m.
- **Unlimited vs. limited torque:** For the limited-torque variants, the actuator saturates at a lower bound, causing the arm to take significantly more steps at $r_{\text{scale}} = 0.001$ (1160 vs. 309 steps). The limited-torque environment also shows a different convergence pattern: the mean error is lower (8.38 vs. 19.57) because the smoother torque profile avoids large transient deviations.
- **v0 vs. v1 variants:** The v0 and v1 variants produce nearly identical trajectories for the unlimited-torque case (same steps, same error), while showing minor differences in total reward for the limited-torque case due to different reward scaling.
- **Trade-off:** There is a clear speed–aggressiveness trade-off. Smaller r_{scale} yields faster convergence but demands higher torques and produces larger transient tracking errors. For practical applications, $r_{\text{scale}} = 0.1$ offers a good balance: sub-second convergence (469–1007 steps) with moderate torques.

2.3 Speed Up iLQR

A key practical question is how the control cost weight r_{scale} affects convergence speed. We ablate 10 log-spaced values of $r_{\text{scale}} \in [0.001, 1.0]$ on both **TwoLinkArm-v0** (unlimited torque) and **TwoLinkArm-limited-torque-v0**. Table 4 reports the runtime, convergence steps, and control metrics.

Table 4: iLQR ablation over r_{scale} for two environments (SIM_TIME=4 s).

Env	r_{scale}	Time (s)	Steps	Reward	Final Err	Mean Err	Mean $\ u\ $	Max $\ u\ $	Settled
v0	0.001	5.6	309	-695	0.010	19.6	37.4	225.0	5.8%
v0	0.005	5.7	399	-709	0.010	15.2	28.3	218.5	25.6%
v0	0.01	5.4	416	-725	0.010	14.7	26.4	211.6	26.0%
v0	0.05	5.6	507	-842	0.013	12.4	18.1	171.0	16.6%
v0	0.1	5.9	469	-1487	0.012	13.2	15.7	103.9	2.4%
v0	0.2	10.4	817	-2290	0.012	7.5	6.9	72.3	25.0%
v0	0.5	12.8	1914	-6748	0.010	3.1	2.0	16.0	12.2%
v0	1.0	20.7	3800	-12721	0.011	2.0	0.9	11.6	13.8%
LT-v0	0.001	15.0	1160	-6844	0.010	8.4	49.6	225.0	3.1%
LT-v0	0.005	12.4	1171	-6609	0.010	7.7	46.8	218.5	3.0%
LT-v0	0.01	12.9	1163	-6231	0.011	7.2	45.0	211.6	3.1%
LT-v0	0.05	13.6	1049	-4786	0.012	5.5	34.9	171.0	3.5%
LT-v0	0.1	12.7	1007	-4368	0.011	4.9	20.9	133.1	6.2%
LT-v0	0.2	10.1	1095	-3641	0.010	5.0	6.7	60.7	23.1%
LT-v0	0.5	13.0	1931	-6939	0.010	3.1	1.9	16.0	12.4%
LT-v0	1.0	20.7	3800	-12751	0.011	2.0	0.9	11.6	13.7%

Why smaller r_{scale} accelerates convergence The mechanism is straightforward: r_{scale} controls the penalty on $\|u\|^2$ in the intermediate cost. A smaller weight allows the planner to use larger torques, which produces faster joint accelerations and reaches the goal in fewer timesteps. Since each iLQR iteration processes all $N = 100$ horizon steps, and the MPC replans every 50 simulation steps, fewer total simulation steps directly means fewer replanning calls and less wall-clock time.

In **TwoLinkArm-v0** (unlimited torque), this effect is dramatic: reducing r_{scale} from 1.0 to 0.001 cuts convergence from 3800 to 309 steps (92% reduction) and runtime from 20.7 s to 5.6 s (73% reduction). The solver finds a near-bang-bang solution that drives the arm to the goal in under 0.3 s.

In **TwoLinkArm-limited-torque-v0**, the actuator saturates at a fixed bound, so even with $r_{\text{scale}} = 0.001$ the arm cannot exceed the torque limit. The convergence steps remain ~ 1000 – 1200 for $r_{\text{scale}} \leq 0.2$, and only increase for $r_{\text{scale}} \geq 0.5$ when the penalty becomes so large that the solver avoids using the available torque. This saturation effect explains why the limited-torque environment shows a much weaker dependence on r_{scale} : the physical constraint, not the cost function, is the binding bottleneck.

Trade-off While smaller r_{scale} yields faster convergence, it comes at the cost of higher peak torques (225 N·m at $r_{\text{scale}} = 0.001$ vs. 12 N·m at $r_{\text{scale}} = 1.0$) and larger transient tracking errors (mean error 19.6 vs. 2.0). The reward also degrades because the environment’s reward function penalizes both state error and control effort. For practical deployment, $r_{\text{scale}} \in [0.01, 0.1]$ offers the best balance: sub-second convergence with moderate torques and good tracking.

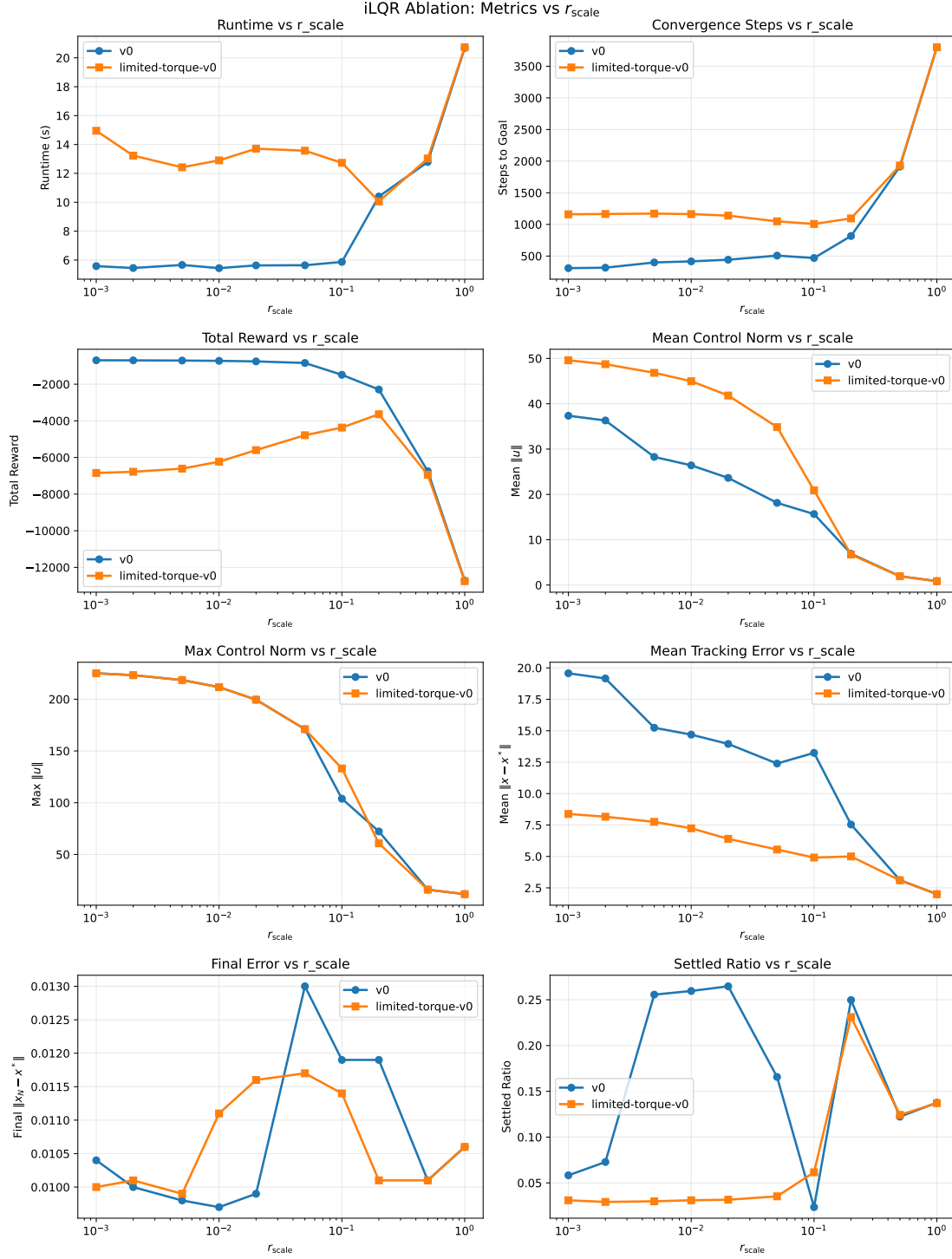


Figure 6: Ablation metrics vs. r_{scale} for TwoLinkArm-v0 and TwoLinkArm-limited-torque-v0. Top row: runtime and convergence steps. Middle row: total reward and mean control norm. Bottom row: max control norm and mean tracking error.

A PID Controller Core Implementation

The following code shows the core computation logic of the dual-loop PID controller:

Listing 1: CartPolePIDController.compute method

```
1 def compute(self, observation, dt):
2     x, x_dot, theta, theta_dot = observation[:4]
3
4     # Angle PD + I
5     theta_error = theta
6     self._theta_integral += theta_error * dt
7     self._theta_integral = np.clip(
8         self._theta_integral, -self.integral_limit, self.integral_limit
9     )
10    f_angle = (
11        self.theta_kp * theta_error
12        + self.theta_kd * theta_dot
13        + self.theta_ki * self._theta_integral
14    )
15
16    # Cart PD + I
17    x_error = x
18    self._x_integral += x_error * dt
19    self._x_integral = np.clip(
20        self._x_integral, -self.integral_limit, self.integral_limit
21    )
22    f_cart = (
23        self.x_kp * x_error
24        + self.x_kd * x_dot
25        + self.x_ki * self._x_integral
26    )
27
28    # Total force
29    force = f_angle + f_cart
30
31    # Hysteresis deadband
32    if abs(force) < self.hysteresis:
33        force = 0.0
34
35    # Clip and map to discrete action
36    force = np.clip(force, -self.force_limit, self.force_limit)
37    return 1 if force >= 0 else 0
```

B iLQR Trajectory Plots

This appendix contains the detailed trajectory plots for all 12 iLQR configurations. Each figure shows 8 metrics: joint position tracking, joint velocity, control torque, tracking error (log scale), instantaneous control effort, cumulative control effort, system energy, and per-step reward.

B.1 $r_{\text{scale}} = 0.5$

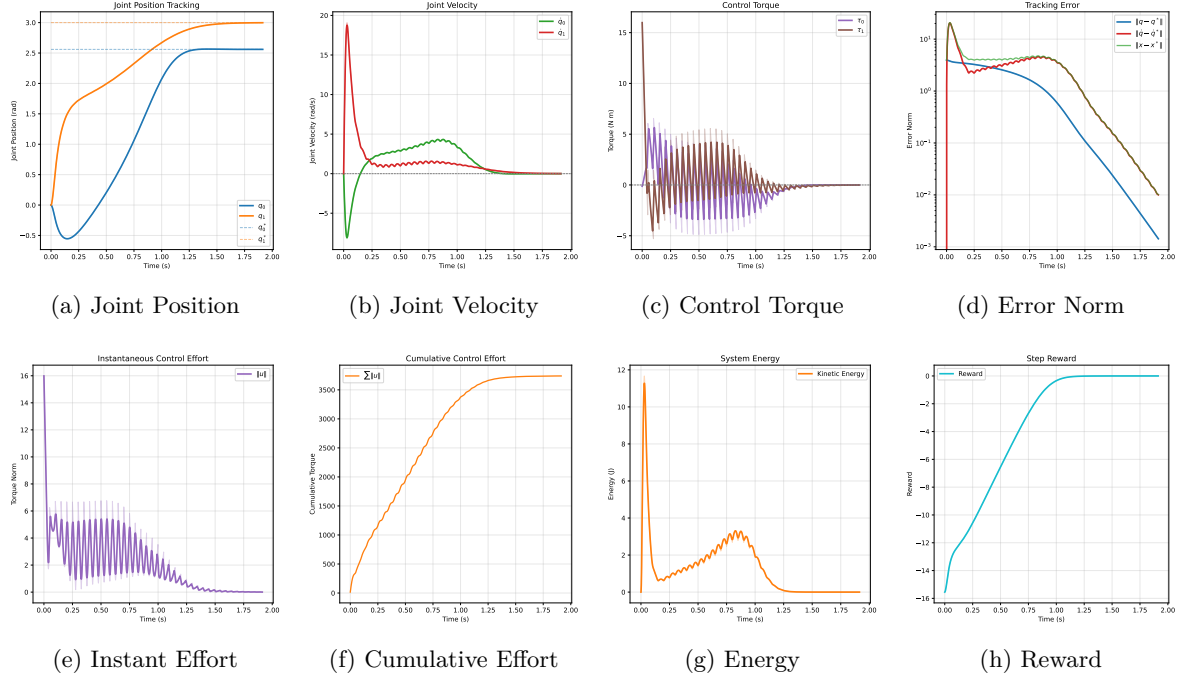


Figure 7: iLQR trajectory for **TwoLinkArm-v0**, $r_{\text{scale}} = 0.5$.

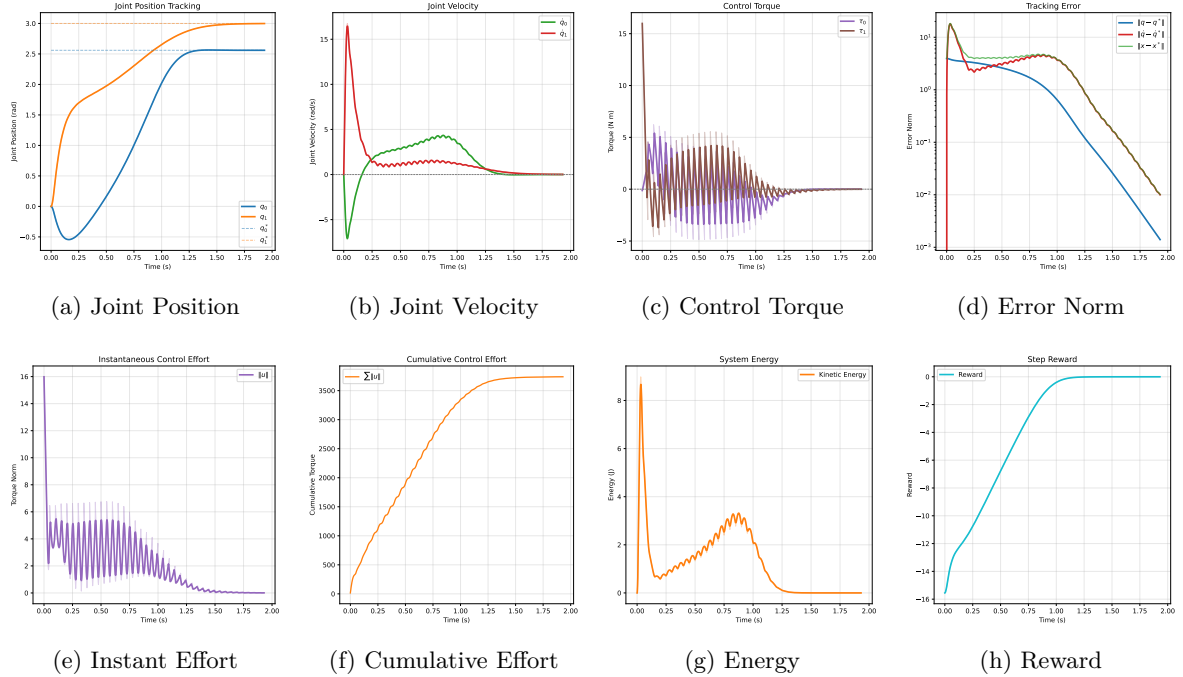


Figure 8: iLQR trajectory for **limited-torque-v0**, $r_{\text{scale}} = 0.5$.

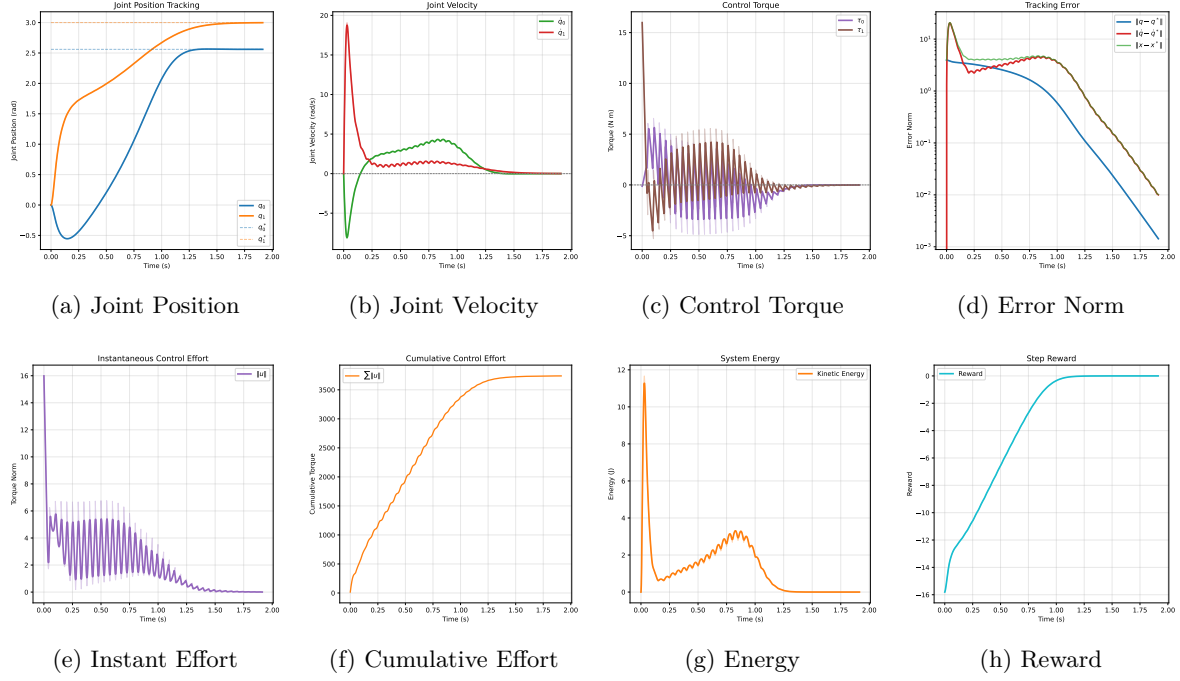


Figure 9: iLQR trajectory for **TwoLinkArm-v1**, $r_{\text{scale}} = 0.5$.

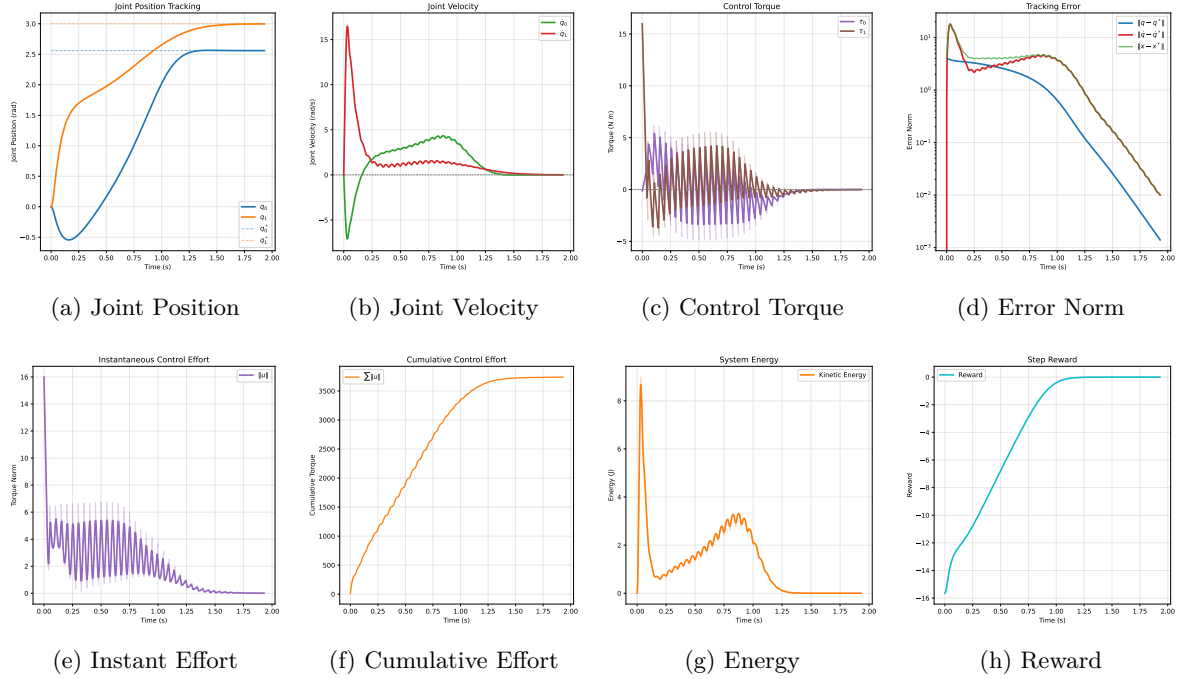


Figure 10: iLQR trajectory for **limited-torque-v1**, $r_{\text{scale}} = 0.5$.

B.2 $r_{\text{scale}} = 0.1$

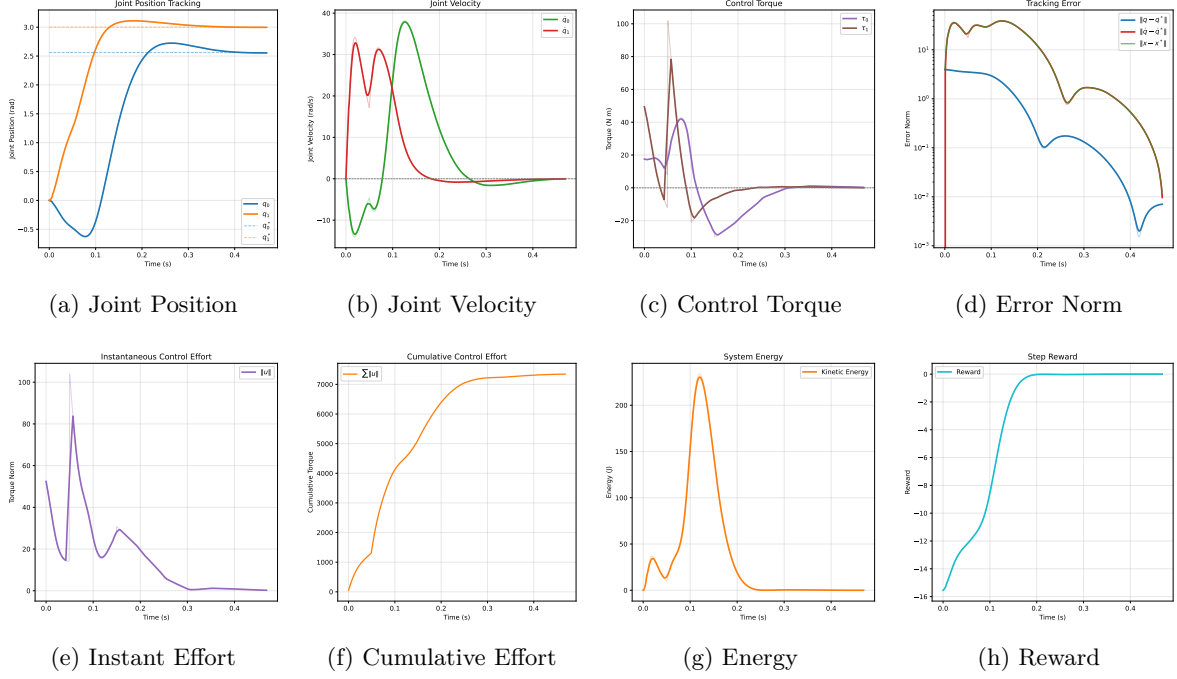


Figure 11: iLQR trajectory for **TwoLinkArm-v0**, $r_{\text{scale}} = 0.1$.

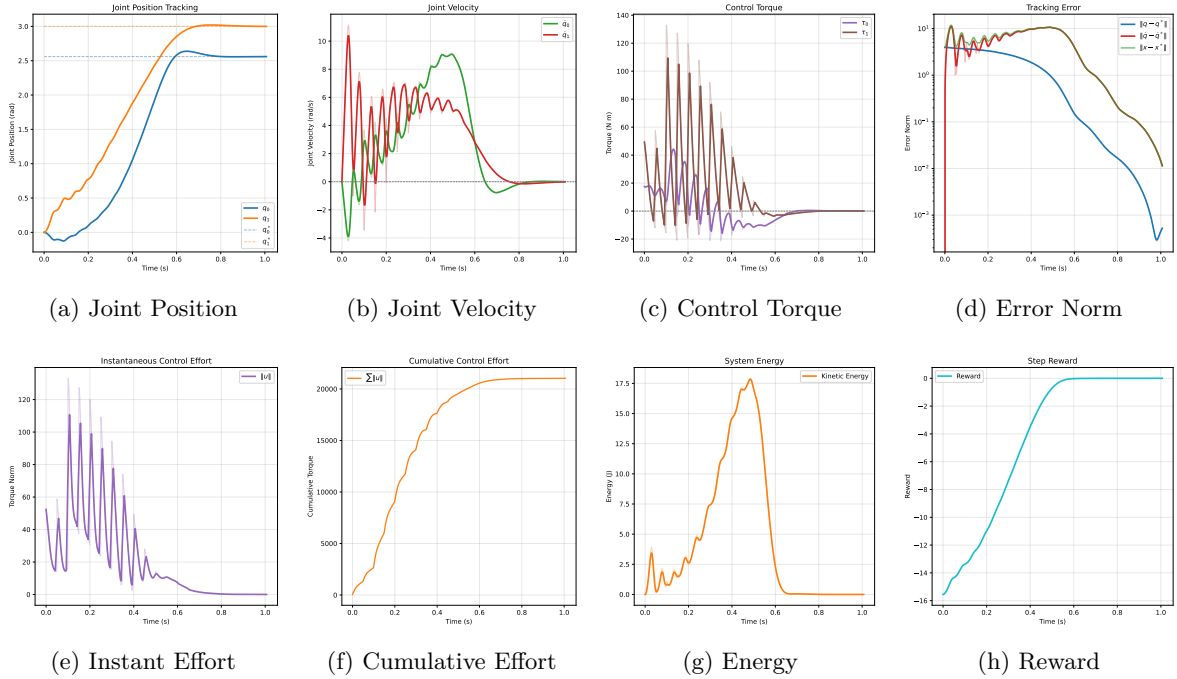


Figure 12: iLQR trajectory for **limited-torque-v0**, $r_{\text{scale}} = 0.1$.

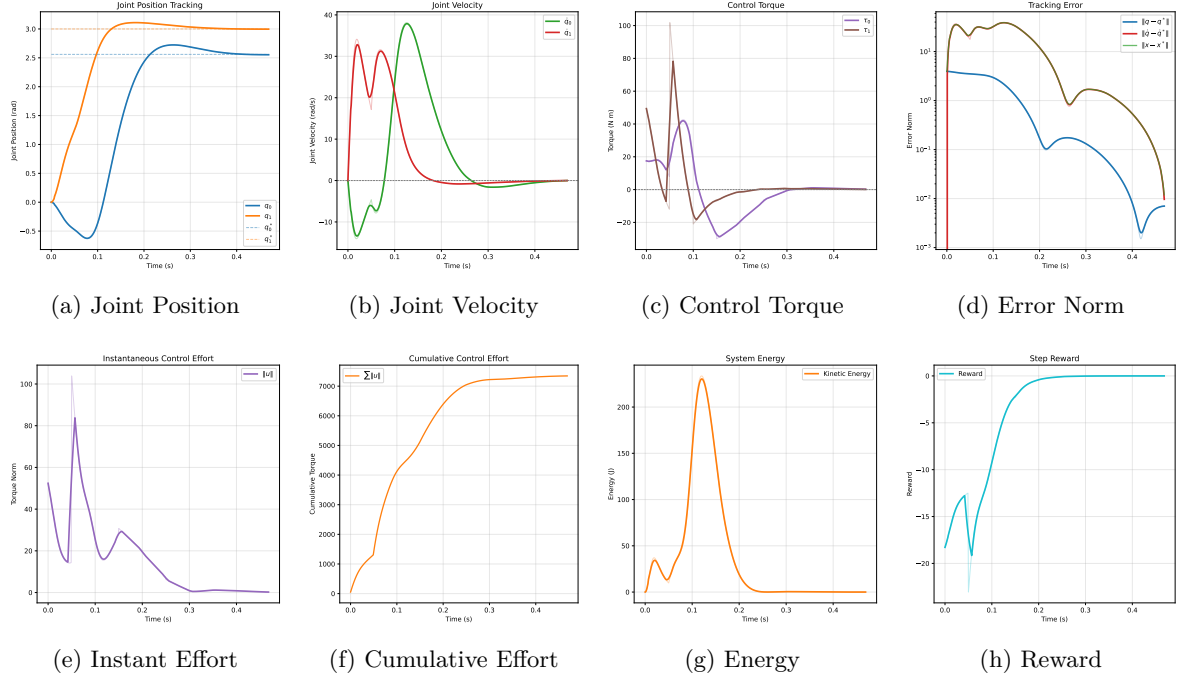


Figure 13: iLQR trajectory for **TwoLinkArm-v1**, $r_{\text{scale}} = 0.1$.

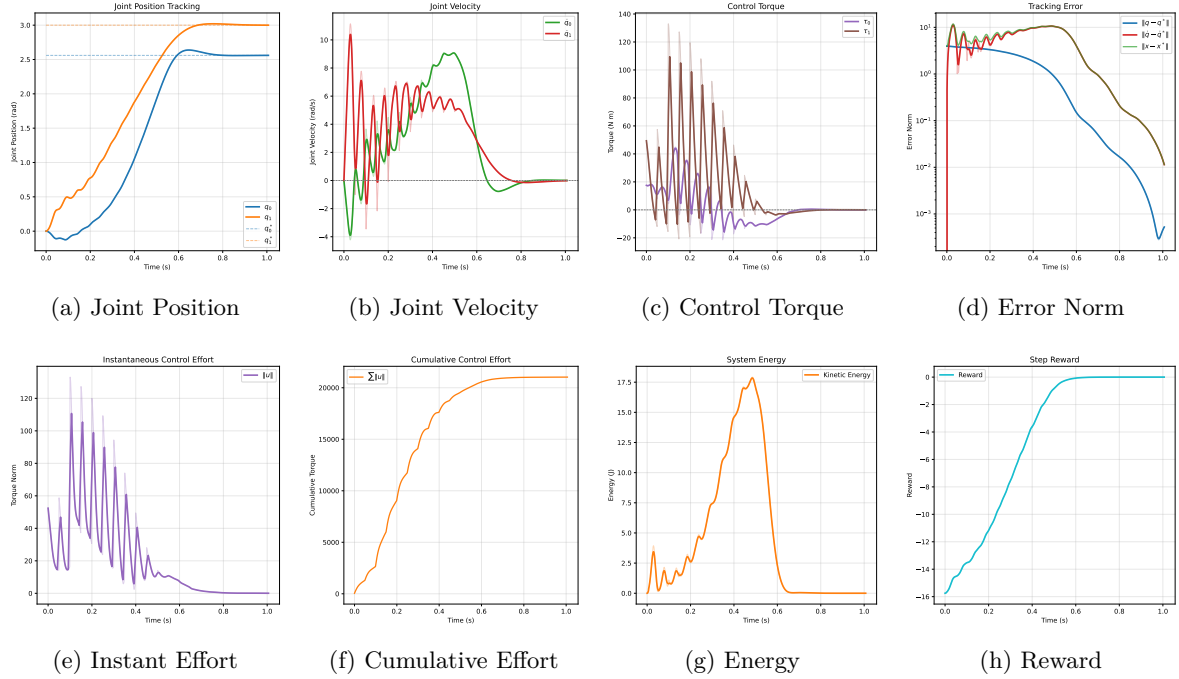


Figure 14: iLQR trajectory for **limited-torque-v1**, $r_{\text{scale}} = 0.1$.

B.3 $r_{\text{scale}} = 0.001$

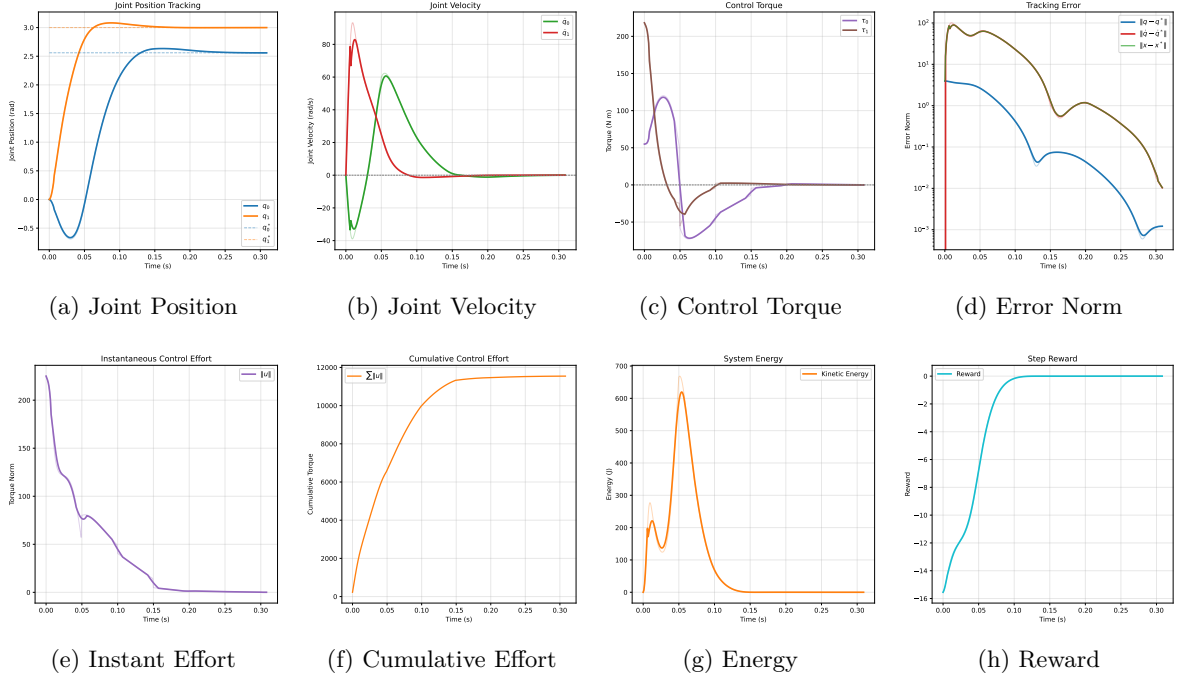


Figure 15: iLQR trajectory for **TwoLinkArm-v0**, $r_{\text{scale}} = 0.001$.

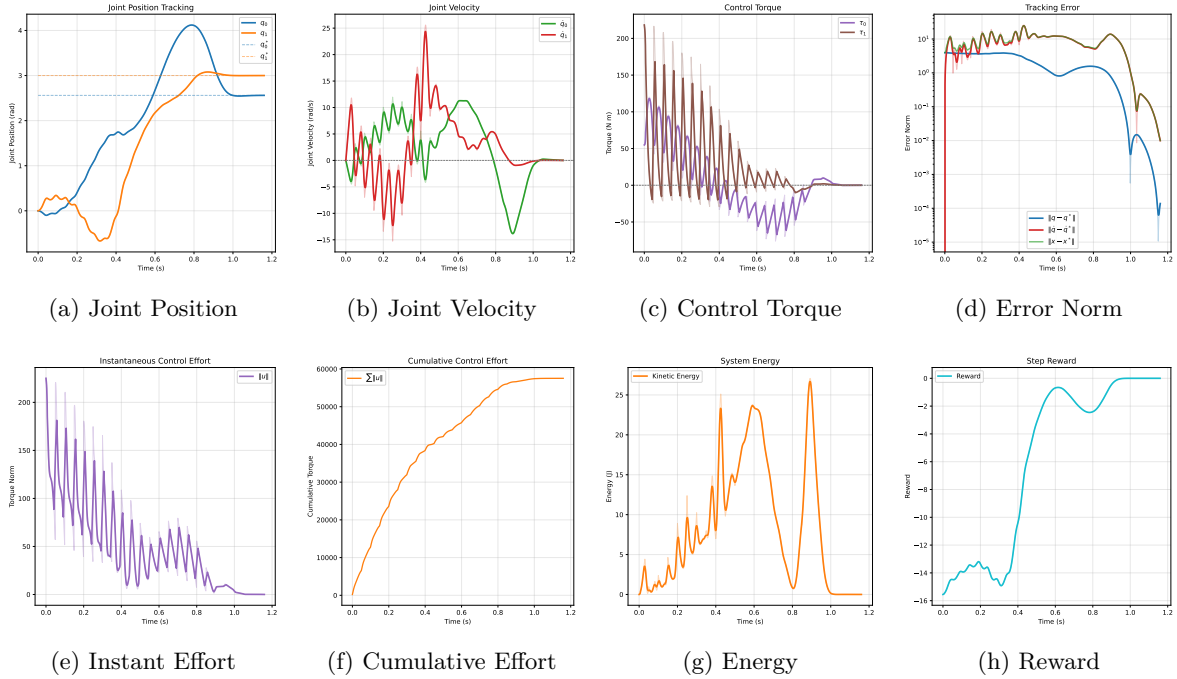


Figure 16: iLQR trajectory for **limited-torque-v0**, $r_{\text{scale}} = 0.001$.

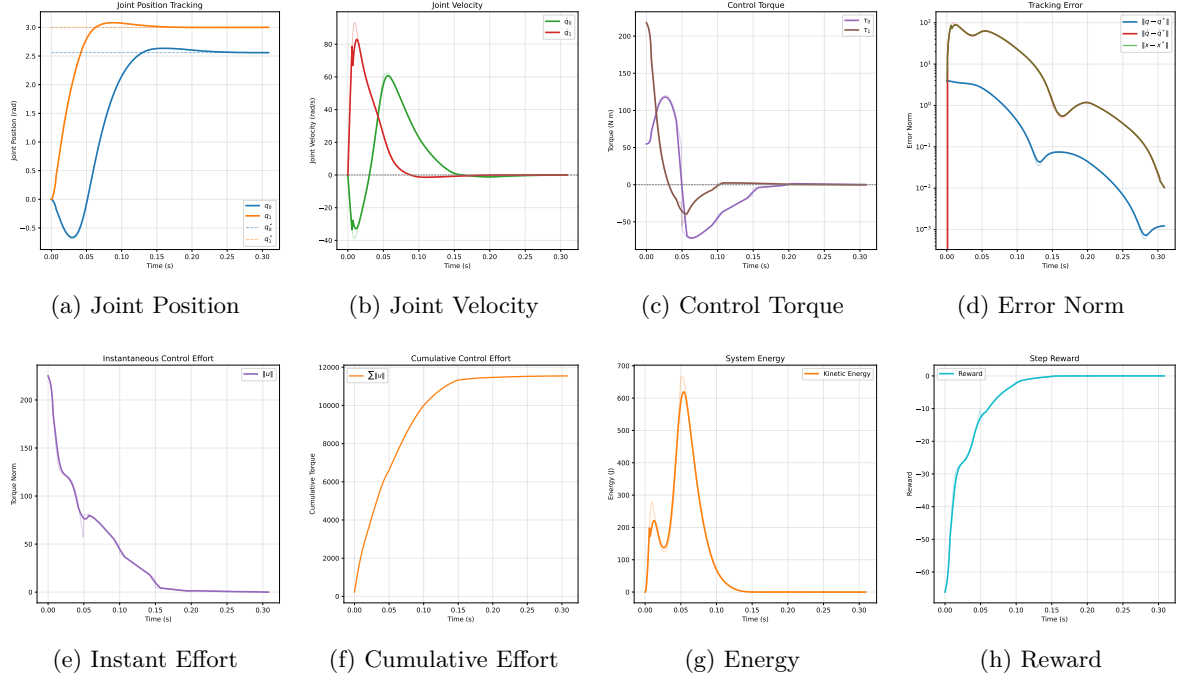


Figure 17: iLQR trajectory for **TwoLinkArm-v1**, $r_{\text{scale}} = 0.001$.

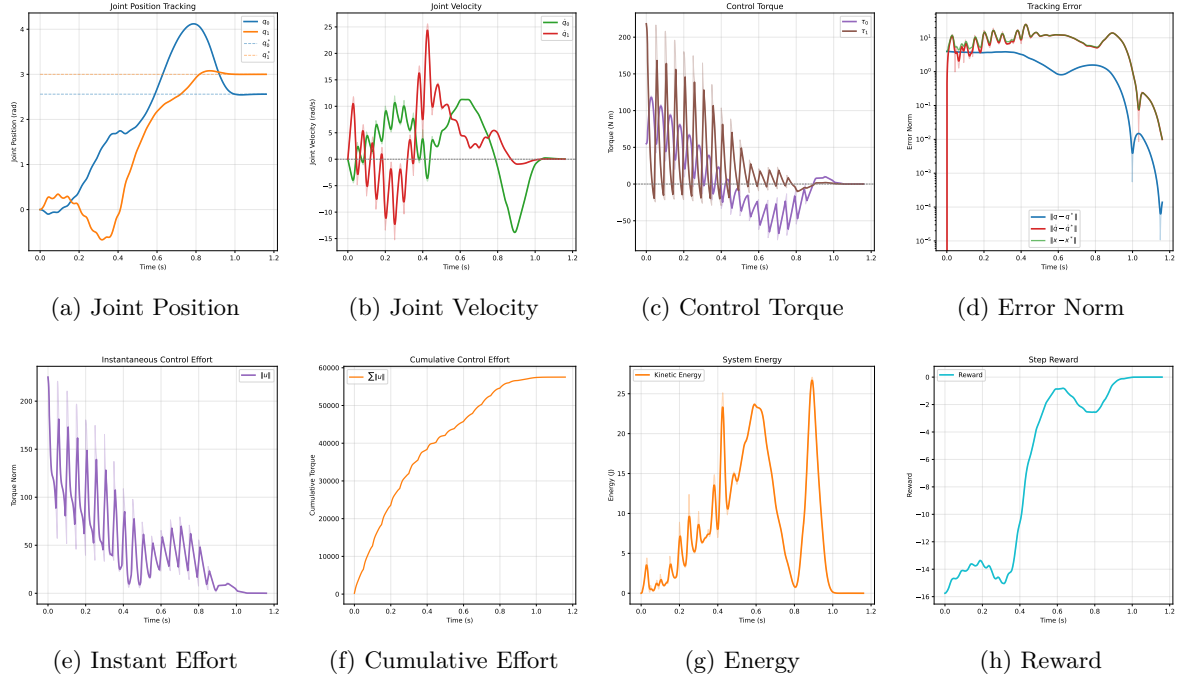


Figure 18: iLQR trajectory for **limited-torque-v1**, $r_{\text{scale}} = 0.001$.